Prerequisites

Create a Microsoft Foundry project

Deploy a model

Get the endpoint

Create a client application to chat with the model

Summary

Clean up

# Create a generative AI chat app

In this exercise, you use the OpenAI SDK and the Responses API to create a chat app that connects to a model deployed in a Microsoft Foundry project.

This exercise takes approximately **45** minutes.

> **Note**: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

## Prerequisites

Before starting this exercise, ensure you have:

- An active Azure subscription
- Visual Studio Code installed
- Python version **3.13.xx** installed*
- Git installed and configured
- Azure CLI installed

> * Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

## Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open the Microsoft Foundry portal at `https://ai.azure.com` to start building; signing in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in.
2. If it is not already enabled, in the tool bar at the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
    - **Foundry resource**: *Use the default name for your resource (usually {project_name}-resource)*
    - **Subscription**: *Your Azure subscription*
    - **Resource group**: *Create or select a resource group*
    - **Region**: Select any of the **AI Foundry recommended** regions in **this list**
3. Wait for your project to be created. Then view its home page.

## Deploy a model

Next, let's deploy a model that you'll use in your chat application.

1. Now you're ready to explore models. On the **Discover** page, select the **Models** tab to view the Microsoft Foundry model catalog.
2. In the model catalog, search for `gpt-5.2`.
3. Review the model card, and then deploy it using the default settings.
4. When the model has been deployed, it will open in the model playground - you can test it there if you like.

# Get the endpoint

You'll need an endpoint to connect to the model from a client application. In this exercise, we're going to use the OpenAI SDK to chat with the model; and we'll use the Azure OpenAI endpoint with Entra ID authentication to connect to it.

> **Note**: As an alternative to Entra ID authentication, you could use the API Key for the project. using Entra ID authentication is preferred whenever possible.

1. On the menu bar, select the **Home** page.
2. Note the **Azure OpenAI Endpoint** displayed there.

> **Tip**: You'll use the **Azure OpenAI Endpoint** in this exercise, not the project endpoint!

# Create a client application to chat with the model

Now that you have deployed a model, you can use the OpenAI SDK and the Responses API to develop an application that chats with it.

## Get the application files from GitHub

The initial application files you'll need to develop your chat application are provided in a GitHub repo.

1. Open Visual Studio Code.
2. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-studio` repo to a local folder (it doesn't matter which one). Then open it.

   You may be prompted to confirm you trust the authors.

## Prepare the application configuration

1. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
2. In the **Command Palette**, use the command `python:select interpreter`. Then create a new **Venv** environment based on your Python 3.13 installation.

> **Tip**: If you are prompted to install dependencies, you can install the ones in the *requirements.txt* file in the */labfiles/foundry-chat/python/chat-app* folder; but it's OK if you don't - we'll install them later!

3. In the Explorer pane, navigate to the folder containing the application code files at **/labfiles/foundry-chat/python/chat-app**. The application files include:
   - **.env** (the application configuration file)
   - **requirements.txt** (the Python package dependencies that need to be installed)
   - **chat-app.py** (the code file for the chat application)
   - **chat-async.py** (the code file for an asynchronous version of the application)
4. In the **Explorer** pane, right-click the **chat-app** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the */labfiles/foundry-chat/python/chat-app* folder.)

> **Note**: Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

5. Ensure that the terminal is open in the **labfiles/foundry-chat/python/chat-app** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.

6. Install the OpenAI SDK, Azure Identity, and other required packages by running the following command:

```
pip install -r requirements.txt
```

7. In the **Explorer** pane, in the **labfiles/foundry-chat/python/chat-app** folder, select the **.env** file to open it. Then update the configuration values to include the **Azure OpenAI Endpoint** and the name assigned to the deployment for the **gpt-5.2** model.

> **Tip**: Copy the **Azure OpenAI Endpoint** (not the project endpoint!) from the project home page in the Foundry portal, and enter the exact deployment name assigned to your deployment in the `MODEL_DEPLOYMENT` setting.

Save the modified configuration file.

## Use the *ChatCompletions* API to chat with the model

The *ChatCompletions* API is a well-established way to build client applications for large language models, and has been widely adopted.

1. In the **Explorer** pane, in the **labfiles/foundry-chat/python/chat-app** folder, select the **chat-app.py** file (not *chat-async.py*) to open it.

2. Review the existing code. You will add code to use the OpenAI SDK to access your model.

> **Tip**: As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespace you will need to use the OpenAI SDK:

```
# import namespaces
from openai import OpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
```

4. In the **main** function, note that code to load the endpoint and key from the configuration file has already been provided. Then find the comment **Initialize the OpenAI client**, and add the following code to create a client for the OpenAI API:

```
# Initialize the OpenAI client
token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default"
)

openai_client = OpenAI(
    base_url=azure_openai_endpoint,
    api_key=token_provider
)
```

5. In the **main** function, note that code to request a user prompt until the user quits the app has been provided. Within this loop, find the **Get a response** comment, and add the following code:

Code Copy

```
# Get a response
completion = openai_client.chat.completions.create(
     model=model_deployment,
     messages=[
         {
             "role": "system",
             "content": "You are a helpful AI assistant that answers questions and provides
information."
         },
         {
             "role": "user",
             "content": input_text
         }
     ]
)
print(completion.choices[0].message.content)
```

Note that the *ChatCompletions* API uses a JSON collection of *messages* to encapsulate the conversation. Often, these consist of a *system prompt* that provides instructions to the model, and a *user prompt* that includes the user's input.

6. Save the changes to the code file. Then, in the terminal pane, use the following command to sign into Azure.

Code Copy

```
az login
```

> **Note**: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *–tenant* parameter. See Sign into Azure interactively using the Azure CLI for details.

7. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

8. After you have signed in, enter the following command to run the application:

Code Copy

```
python chat-app.py
```

The program should run in the terminal (if not, resolve any errors and try again).

9. When prompted, enter the following prompt:

Code Copy

```
Tell me about the ELIZA chatbot.
```

After a few moments, the app should respond with some information about the ELIZA chatbot created in the 1960s.

10. Enter the prompt `quit` to end the application.

## Use the *Responses* API to chat with the model

While the *ChatCompletions* API is widely used, it is increasingly being superseded by the newer *Responses* API. Let's update the code to use it.

1. In the **chat-app.py** code, in the **main** function, replace the code under the comment **Get a response** with the following code that uses the *Responses* API.

```
Code
Copy
```

```
# Get a response
response = openai_client.responses.create(
            model=model_deployment,
            instructions="You are a helpful AI assistant that answers questions and
provides information.",
            input=input_text
)
print(response.output_text)
```

   Note the simpler syntax in which the system message is assigned to the *instructions* parameter, and the user prompt is assigned to the *input* parameter.
2. Save the changes to the code, and in the terminal pane, re-run the application ( `python chat-app.py` ).
3. When prompted, enter the same prompt as before:

```
Code
Copy
```

```
Tell me about the ELIZA chatbot.
```

   After a few moments, the app should once again respond with some information about the ELIZA chatbot.
4. Enter the following prompt to try to continue the conversation:

```
Code
Copy
```

```
How does it compare to modern LLMs?
```

   The app should respond in a way that indicates it doesn't understand what "it" refers to. The conversation context has been lost. We'll fix that.
5. Enter the prompt `quit` to end the application.

## Add conversation tracking

To maintain the conversational context, we need to include references to previous responses in each new request.

1. In the **chat-app.py** code, in the **main** function, find the comment **Loop until the user wants to quit**, and add the following code above it (*before* the loop):

```
Code
Copy
```

```
# Track responses
last_response_id = None
```

2. Modify the code under the comment **Get a response** with the following code to pass the previous response ID on the request, and then obtain the new response ID so it can be added next time.

```
Code
Copy
```

```
# Get a response
response = openai_client.responses.create(
            model=model_deployment,
            instructions="You are a helpful AI assistant that answers questions and
provides information.",
            input=input_text,
            previous_response_id=last_response_id,
)
print(response.output_text)
last_response_id = response.id
```

Using this technique, you can pass the ID of the previous response to maintain context. You could also implement more complex logic to pass an ID from any previous response to redirect a conversation or resume a previous conversational thread.

3. Save the changes to the code, and in the terminal pane, re-run the application ( `python chat-app.py` ).
4. When prompted, enter the same prompt as before:

Code Copy

```
Tell me about the ELIZA chatbot.
```

After a few moments, the app should once again respond with some information about the ELIZA chatbot.

5. Enter the following prompt to try to continue the conversation:

Code Copy

```
How does it compare to modern LLMs?
```

This time, the app should respond with a comparison of the ELIZA chatbot and modern LLMs. The response may be quite lengthy, and the app waits until it has all been received from the model before displaying it, which may make the app seem unresponsive. We'll fix that next!

6. Enter the prompt `quit` to end the application.

## Implement *streaming* responses

To handle long responses, you can use *streaming* to start processing partial responses before the full text has been returned.

1. In the **chat-app.py** code, in the **main** function, replace the code under the comment **Get a response** with the following code that uses *streaming*.

Code Copy

```
# Get a response
stream = openai_client.responses.create(
            model=model_deployment,
            instructions="You are a helpful AI assistant that answers questions and
provides information.",
            input=input_text,
            previous_response_id=last_response_id,
            stream=True
)
for event in stream:
     if event.type == "response.output_text.delta":
         print(event.delta, end="")
     elif event.type == "response.completed":
         last_response_id = event.response.id
print()
```

Note that the *stream=True* parameter creates a streamed response in which *events* occur as each new chunk (or *delta*) is ready for processing.

2. Save the changes to the code, and in the terminal pane, re-run the application ( `python chat-app.py` ).

3. When prompted, enter the same prompt as before:

   Code Copy

   ```
   Tell me about the ELIZA chatbot.
   ```

   After a few moments, the app should start responding with some information about the ELIZA chatbot. The response should appear incrementally as each chunk is returned.

4. Enter the following prompt to try to continue the conversation:

   Code Copy

   ```
   How does it compare to modern LLMs?
   ```

   Again, the response should be displayed incrementally.
5. Enter the prompt `quit` to end the application.

## Use the asynchronous API

The OpenAI SDK offers an asynchronous option that can increase the responsiveness of applications when using long-running model or agent operations.

1. In the **Explorer** pane, in the **labfiles/foundry-chat/python/chat-app** folder, select the **chat-async.py** file (not *chat-app.py*) to open it.

2. Review the existing code. You will add code to use the OpenAI SDK async API to access your model.

   > **Tip**: As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespace you will need to use the OpenAI SDK:

   Code Copy

```
# import namespaces for async
import asyncio
from openai import AsyncOpenAI
from azure.identity.aio import DefaultAzureCredential, get_bearer_token_provider
```

4. In the **main** function, note that code to load the endpoint and key from the configuration file has already been provided. Then find the comment **Initialize an async OpenAI client**, and add the following code to create a client for the OpenAI API:

Code Copy

```
# Initialize an async OpenAI client
credential = DefaultAzureCredential()
token_provider = get_bearer_token_provider(
 credential, "https://ai.azure.com/.default"
)

async_client = AsyncOpenAI(
     base_url=azure_openai_endpoint,
     api_key=token_provider
)
```

5. In the **main** function, note that code to request a user prompt until the user quits the app has been provided. Within this loop, find the **Await an asynchronous response** comment, and add the following code:

Code Copy

```
# Await an asynchronous response
response = await async_client.responses.create(
             model=model_deployment,
             instructions="You are a helpful AI assistant that answers questions and
provides information.",
             input=input_text,
             previous_response_id=last_response_id
)
assistant_text = response.output_text
print("Assistant:", assistant_text)
last_response_id = response.id
```

This code awaits an asynchronous response from the model.

6. At the end of the **main** function, in the **finally** block, find the comment **Close the async client session** and add the following code to close the asynchronous client:

Code Copy

```
# Close the async client session
 await credential.close()
```

7. Save the changes to the code file. Then, in the terminal pane, use the following command to run the program:

Code Copy

```
python chat-async.py
```

The program should run in the terminal (if not, resolve any errors and try again).

8. When prompted, enter the following prompt:

```
Tell me about the Turing test.
```

After a few moments, the app should respond with some information about the Turing test.

9. Enter the prompt `quit` to end the application.

## Summary

In this exercise, you used the OpenAI SDK and the *ChatCompletions* and *Responses* APIs to create a client application for a generative AI model that you deployed in a Microsoft Foundry project. You customized the model's behavior by tracking conversational context and implemented streaming to deliver a responsive chat experience.

## Clean up

If you've finished exploring Microsoft Foundry, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

1. Open the Azure portal and view the contents of the resource group where you deployed the resources used in this exercise.
2. On the toolbar, select **Delete resource group**.
3. Enter the resource group name and confirm that you want to delete it.